

9. Recent Trends

(3 hours, 4 marks)

By Bidur Sapkota

9.1 DevOps and Continuous Deployment Pipelines

DevOps is a set of practices, cultural philosophies, and tools that unify software development (Dev) and IT operations (Ops) to shorten the systems development life cycle and deliver high-quality software continuously. The term was coined around 2008 to 2009 by Patrick Debois and Andrew Shafer, and has since become a dominant paradigm in modern software engineering.

Traditionally, development teams and operations teams worked in silos. Developers wrote code and "threw it over the wall" to operations for deployment, leading to blame games, slow releases, and fragile production environments. DevOps breaks down these silos by encouraging shared ownership, collaboration, and automation across the entire software delivery pipeline.

Principles of DevOps

- 1. Culture of collaboration:** Developers and operations staff share responsibility for the entire lifecycle, from development through production. There is no "your problem" when something breaks; the team collectively owns quality and uptime.
- 2. Automation:** Manual, error-prone tasks (building, testing, deploying, provisioning infrastructure) are automated to achieve speed, consistency, and repeatability.
- 3. Continuous feedback:** Monitoring, logging, and alerting systems provide real-time feedback on application health and performance, allowing teams to detect and resolve issues quickly.

Principles of DevOps

- 4. Continuous improvement:** Teams regularly reflect on processes and outcomes, iterating on both the software and the delivery pipeline itself.
- 5. Infrastructure as Code (IaC):** Infrastructure is defined and managed through code (configuration files) rather than manual setup, enabling version control, repeatability, and auditability.

DevOps Lifecycle

The DevOps lifecycle is often depicted as an infinity loop (∞) representing the continuous, iterative nature of the process:

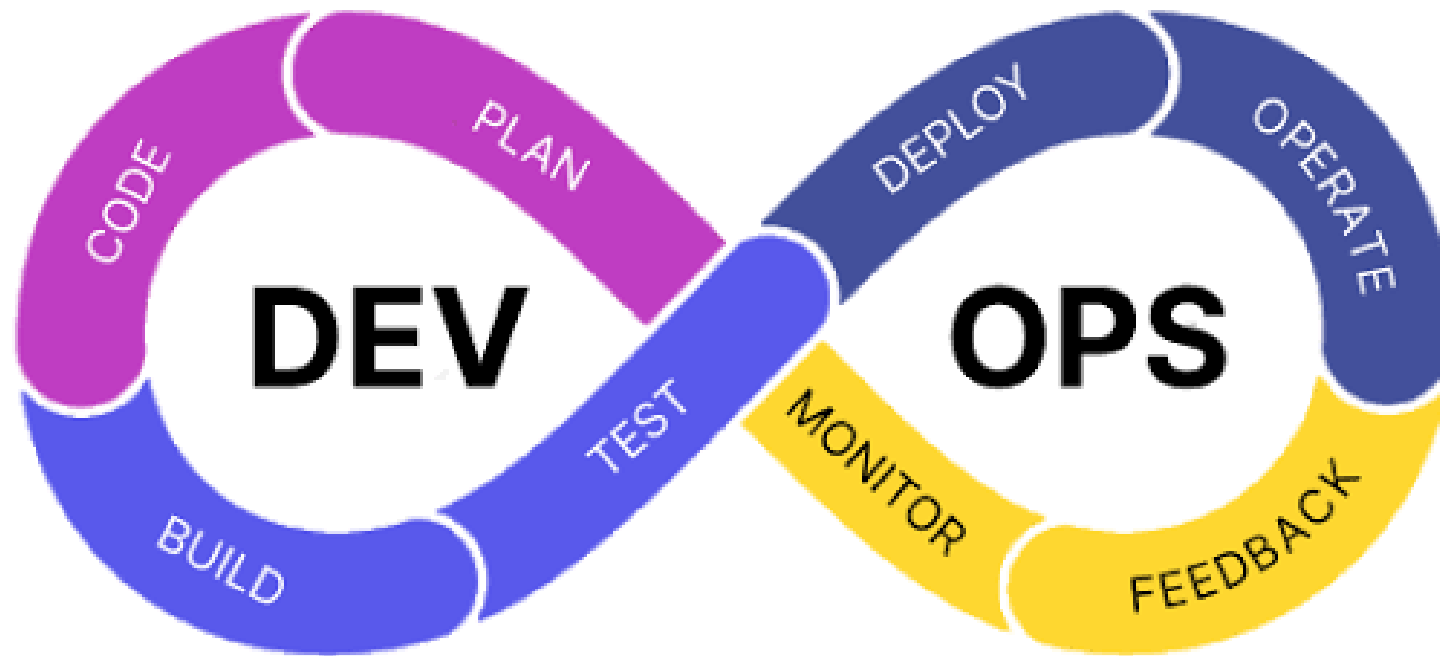
- 1. Plan:** Define requirements, user stories, and sprint goals.
- 2. Code:** Develop features in short iterations using version control.
- 3. Build:** Compile and package the application into deployable artifacts.
- 4. Test:** Run automated tests (unit, integration, system) to validate quality.
- 5. Release:** Approve and tag a production-ready version.

DevOps Lifecycle

- 6. Deploy:** Push the release to production environments (automatically or with approval).
- 7. Operate:** Manage and maintain the live application and its infrastructure.
- 8. Monitor:** Collect metrics, logs, and traces to observe application behavior and user experience.

The cycle then feeds back into the Plan phase, creating a continuous loop.

DevOps Lifecycle



Continuous Deployment Pipeline

A continuous deployment pipeline automates the journey of code from a developer's commit to production. It extends beyond CI/CD by removing all manual gates. Every change that passes automated validation is automatically deployed to production without human intervention.

Stages of a deployment pipeline:

- 1. Source stage:** A developer pushes code to the version control repository (e.g., Git). This triggers the pipeline automatically.
- 2. Build stage:** The source code is compiled, dependencies are resolved, and a deployable artifact (binary, container image, package) is produced.

Continuous Deployment Pipeline

- 3. Test stage:** The artifact is subjected to automated tests such as unit tests, integration tests, security scans, and performance tests. If any test fails, the pipeline halts and the developer is notified.
- 4. Staging/Pre-production stage:** The artifact is deployed to a staging environment that mirrors production. Smoke tests and acceptance tests validate behavior in a production-like setting.
- 5. Production deployment:** The artifact is automatically deployed to the live production environment.

Continuous Deployment Pipeline

6. Monitoring and feedback: Post-deployment monitoring detects anomalies. If critical issues arise, automated rollback mechanisms can revert to the previous stable version.

Benefits of DevOps

- 1. Faster time to market:** Automated pipelines reduce lead time from days or weeks to minutes or hours.
- 2. Higher quality:** Automated testing at every stage catches defects early, reducing the cost and impact of bugs.
- 3. Improved reliability:** Consistent, repeatable deployments eliminate "works on my machine" problems.
- 4. Faster recovery:** Monitoring and automated rollback enable rapid detection and resolution of production incidents.
- 5. Better team morale:** Shared ownership and reduced manual toil lead to more engaged and productive teams.

DevOps Tools

- **Version control:** The popular version control tools include Git, GitHub, and GitLab.
- **CI/CD servers:** Common CI/CD servers are Jenkins, GitHub Actions, GitLab CI/CD, and CircleCI.
- **Containerization:** Docker is widely used for containerization.
- **Orchestration:** Kubernetes is the standard tool for orchestration.
- **Configuration management:** Popular configuration management tools are Ansible, Chef, and Puppet.
- **Monitoring:** Teams use tools like Prometheus, Grafana, and Datadog for monitoring.

9.2 CI/CD in Modern Projects

CI/CD is the backbone of modern DevOps practices. It consists of two related but distinct practices: Continuous Integration (CI) and Continuous Delivery/Deployment (CD).

Continuous Integration (CI):

Continuous Integration is the practice of merging all developers' working copies to a shared mainline frequently, ideally multiple times per day. Each integration triggers an automated build and test process to detect errors as quickly as possible. The central idea is that integrating early and often prevents the "integration hell" that occurs when developers work in isolation for long periods and then try to merge large batches of changes.

Practices of CI

1. Maintain a single source repository (version control) where all code resides.
2. Automate the build so that the entire system can be built with a single command.
3. Make the build self-testing where every build runs a comprehensive suite of automated tests.
4. Every developer commits to the mainline at least once per day.
5. Every commit triggers an automated build and test on the CI server.
6. Fix broken builds immediately. A broken build is the highest-priority issue for the team.
7. Keep the build fast so that developers get feedback within minutes, not hours.

9.2 CI/CD in Modern Projects

Continuous Delivery (CD):

Continuous Delivery extends CI by ensuring that the software is always in a releasable state. After passing through the CI pipeline (build and test), the software is automatically deployed to a staging or pre-production environment. The final deployment to production requires a manual approval step where a human decides when to release.

Continuous Deployment:

Continuous Deployment goes one step further than Continuous Delivery. Every change that passes all stages of the automated pipeline is deployed to production automatically, without any manual intervention. There is no human gate between a successful build and production.

9.2 CI/CD in Modern Projects

Difference between Continuous Delivery and Continuous Deployment:

In Continuous Delivery, the code is automatically prepared, built, tested, and staged for release, but a human must manually approve the final push to production. In Continuous Deployment, the entire process from code commit to production deployment is fully automated. No manual approval is required. Continuous Delivery means "can be released at any time"; Continuous Deployment means "is released every time."

Stages of a CI/CD Pipeline

- 1. Source (Trigger):** The pipeline is triggered when a developer commits code to the repository.
- 2. Build:** Source code is compiled, dependencies are fetched, and a build artifact is produced.
- 3. Test:** Automated tests run to verify the code. Unit tests verify individual components, integration tests verify module interactions, and security or functional tests validate the application.
- 4. Package:** The successfully tested artifact is versioned and stored in an artifact repository or container registry.

Stages of a CI/CD Pipeline

- 5. Deploy to staging:** The artifact is deployed to a staging environment for final validation.
- 6. Deploy to production:** The artifact is released to the production environment (manually approved in CD, automatic in continuous deployment).
- 7. Monitor:** Post-deployment monitoring provides feedback on application health, performance, and user behavior.

Benefits of CI/CD in Modern Projects

- 1. Accelerated feedback:** Developers receive immediate notification when integration fails, allowing fixes while the scope of change is small.
- 2. Increased quality:** Building and testing software on every commit provides continuous confidence in quality.
- 3. Reduced risk:** Frequent, small releases are inherently less risky than infrequent, large releases. If something breaks, the change set is small and easy to diagnose.

Benefits of CI/CD in Modern Projects

- 4. Faster delivery:** Automation removes manual bottlenecks, enabling teams to release new features, bug fixes, and improvements rapidly.
- 5. Improved reporting:** CI/CD tools generate metrics (build times, test coverage, deployment frequency, failure rates) that enable continuous process improvement.

CI/CD in Agile and Scrum

CI/CD complements Agile development naturally. In Scrum, each sprint produces a potentially shippable increment. CI/CD automates the process of making that increment truly shippable by continuously building, testing, and validating the product. CI/CD enables Agile teams to practice short iteration cycles with confidence that each iteration produces a releasable product.

9.3 Cloud-Native Architecture

Cloud-native is an approach to building and running applications that fully exploits the advantages of cloud computing. Rather than simply hosting traditional applications on cloud servers (lift-and-shift), cloud-native applications are designed from the ground up to leverage cloud characteristics such as elasticity, scalability, and resilience.

The Cloud Native Computing Foundation (CNCF) defines cloud-native technologies as those that empower organizations to build and run scalable applications in modern, dynamic environments such as public, private, and hybrid clouds.

Characteristics of cloud-native applications

- 1. Designed for the cloud:** Built to take advantage of cloud elasticity, scaling up or down automatically based on demand rather than being sized for peak capacity.
- 2. Containerized:** Application components are packaged in lightweight containers (e.g., Docker) that bundle the application code with its dependencies, ensuring consistent behavior across development, testing, and production environments.
- 3. Dynamically orchestrated:** Containers are managed by orchestration platforms (e.g., Kubernetes) that automate deployment, scaling, load balancing, and self-healing (automatically restarting failed containers).

Characteristics of cloud-native applications

- 4. Microservices-oriented:** The application is decomposed into small, independent services rather than being built as a single monolith.
- 5. API-driven:** Services communicate through well-defined APIs (typically REST or gRPC), enabling loose coupling and independent evolution.

9.3 Microservices Architecture

Microservices architecture is a design approach in which an application is structured as a collection of small, autonomous services, each running in its own process and communicating through lightweight mechanisms (typically HTTP/REST APIs or message queues). Each service is independently deployable and scalable, and is organized around a specific business capability.

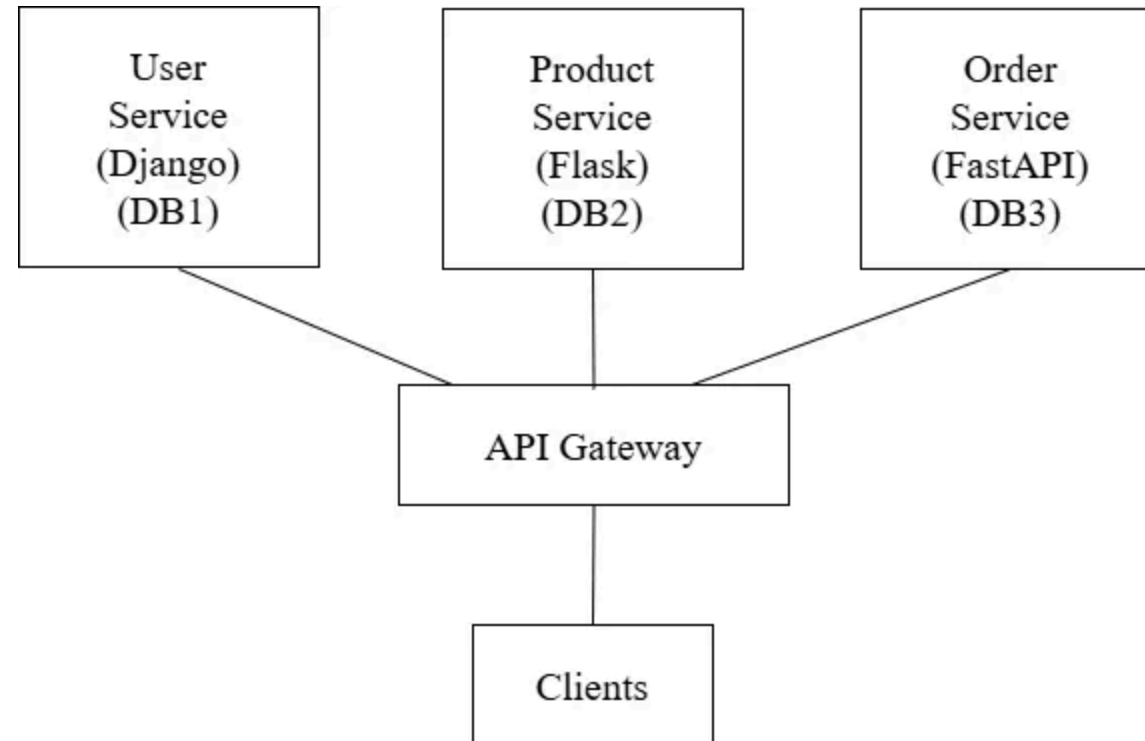
Characteristics of microservices:

1. Single responsibility: Each microservice handles one specific business function, such as user authentication, payment processing, order management, or a notification service.

Characteristics of microservices

- 2. Independent deployment:** Each service can be developed, tested, deployed, and scaled independently without affecting other services.
- 3. Decentralized data management:** Each microservice typically manages its own database or data store, avoiding a single shared database that becomes a bottleneck.
- 4. Technology heterogeneity:** Different services can be built using different programming languages, frameworks, or databases to use the best tool for each specific job.
- 5. Fault isolation:** A failure in one service does not bring down the entire application. Other services continue to function, and the failed service can be restarted independently.

9.3 Microservices Architecture



Monolithic vs. Microservices Architecture

In a monolithic architecture, the entire application is built and deployed as a single unit. All components (UI, business logic, data access) are tightly coupled within one codebase and one deployment artifact. Scaling requires scaling the entire application, even if only one component needs more capacity. A bug in one module can bring down the entire system.

In a microservices architecture, the application is decomposed into independent services. Each service has its own codebase, deployment pipeline, and data store. Services communicate through APIs. Scaling is granular because only the services that need more capacity are scaled. A failure in one service is isolated from the rest.

Advantages of microservices

1. Independent scaling of services based on their individual resource requirements is possible.
2. Faster development and deployment cycles occur because teams can work on different services in parallel.
3. Technology flexibility allows each service to use the technology stack best suited to its function.
4. Better fault isolation and resilience are achieved.
5. Systems are easier to understand and maintain because each service is focused on a single capability.

Challenges of microservices

1. Increased operational complexity occurs as managing dozens or hundreds of services requires sophisticated tooling.
2. Network communication overhead and latency between services can increase.
3. Distributed data management introduces complexity in maintaining consistency across services.
4. Monitoring and debugging are more complex across distributed services.
5. Adopting microservices requires a mature DevOps culture and automation capabilities.

9.3 Infrastructure as Code (IaC)

Infrastructure as Code is the practice of managing and provisioning computing infrastructure (servers, networks, storage, databases, load balancers) through machine-readable configuration files rather than through manual processes or interactive configuration tools.

Instead of an operations engineer manually logging into a cloud console to create servers, configure networking, and set up databases, IaC allows the entire infrastructure to be defined in code files (e.g., Terraform HCL files, YAML, JSON). These files are stored in version control alongside application code, enabling the same software engineering practices such as version control, code review, testing, and CI/CD to be applied to infrastructure management.

Principles of IaC

- 1. Declarative definition:** Infrastructure is described in terms of the desired end state ("I want three web servers behind a load balancer") rather than procedural steps ("first create a server, then install software, then configure the load balancer"). The IaC tool figures out how to achieve the desired state.
- 2. Version control:** Infrastructure definitions are stored in Git, enabling full change history, auditability, and the ability to roll back to any previous infrastructure state.
- 3. Idempotency:** Applying the same configuration multiple times produces the same result, preventing configuration drift and unintended changes.
- 4. Automation:** Infrastructure changes are applied through automated tooling rather than manual commands, eliminating human error and ensuring consistency.

Benefits of IaC

- 5. Consistency:** Every environment (development, staging, production) is provisioned identically from the same code, eliminating configuration drift. This is the gradual divergence between environments that causes "works on my machine" problems.
- 6. Speed:** New environments can be provisioned in minutes rather than days or weeks.
- 7. Reproducibility:** Entire environments can be destroyed and recreated identically, enabling disaster recovery and cost optimization.

Benefits of IaC

- 8. Auditability:** All infrastructure changes are recorded in version control with who made the change, when, and why.
- 9. Cost optimization:** Infrastructure can be provisioned and decommissioned on demand, avoiding the waste of idle resources.

IaC Tools

- **Terraform:** Terraform is an open-source tool by HashiCorp that supports multiple cloud providers with a declarative configuration language.
- **AWS CloudFormation:** AWS CloudFormation is Amazon's native IaC service for defining AWS resources using JSON or YAML templates.
- **Ansible:** Ansible is an agentless configuration management and IaC tool that uses YAML playbooks.
- **Pulumi:** Pulumi is an IaC tool that allows infrastructure definitions in general-purpose programming languages like Python, TypeScript, and Go.

9.4 AI-Assisted Software Development

AI-assisted software development refers to the use of artificial intelligence, particularly large language models and machine learning techniques, to augment and accelerate various phases of the software development lifecycle. Rather than replacing developers, AI tools act as intelligent assistants or AI pair programmers that help developers write code faster, find bugs earlier, generate tests, and produce documentation.

9.4 AI-Assisted Software Development

How AI assists in software development:

1. Code generation and completion:

AI tools analyze the context of the code being written (surrounding code, comments, function signatures) and suggest completions ranging from single lines to entire functions. For example, a developer writing a function signature and a comment describing its purpose may receive a complete, correct implementation suggested by the AI tool. Tools like GitHub Copilot, Amazon CodeWhisperer, and Cursor use LLMs trained on vast code corpora to provide these suggestions.

9.4 AI-Assisted Software Development

How AI assists in software development:

2. Code review and bug detection:

AI-powered tools can automatically review code changes (pull requests) for potential bugs, security vulnerabilities, code style violations, and anti-patterns. They analyze code at a deeper level than traditional linters by understanding code semantics, not just syntax. Tools like CodeRabbit and SonarQube provide automated code review capabilities.

9.4 AI-Assisted Software Development

How AI assists in software development:

3. Test generation:

AI tools can generate unit tests, integration tests, and edge-case test scenarios for existing code. Given a function, the AI can identify boundary conditions, typical inputs, and error cases, and produce test code covering these scenarios. This significantly reduces the manual effort of writing comprehensive test suites.

4. Documentation generation:

AI can generate docstrings, API documentation, README files, and inline comments by analyzing code structure and logic. This helps maintain documentation that stays in sync with the codebase, addressing a common pain point in software projects.

9.4 AI-Assisted Software Development

How AI assists in software development:

5. Natural language to code:

Developers can describe what they want in natural language (e.g., "create a REST endpoint that accepts a JSON payload with name and email fields, validates the email format, and stores the record in the database") and receive working code implementations. This lowers the barrier for less experienced developers and accelerates prototyping.

6. Debugging and error resolution:

AI tools can analyze error messages, stack traces, and code context to suggest fixes for bugs. They can identify root causes that might take a human developer significant time to trace through complex codebases.

9.4 AI-Assisted Software Development

Benefits of AI-assisted development:

- 1. Increased productivity:** There is a significant reduction in time spent on boilerplate code, routine debugging, and scaffolding, which allows developers to focus on higher-level design and problem-solving.
- 2. Faster learning:** Real-time code suggestions help developers learn new languages, frameworks, and APIs by example.
- 3. Improved code quality:** AI-generated tests and automated code review catch issues that might be missed in manual review.
- 4. Reduced context switching:** Developers can get answers and code suggestions within their IDE without switching to documentation or search engines.

9.4 AI-Assisted Software Development

Challenges and risks:

- 1. Code quality concerns:** AI-generated code may contain subtle bugs, security vulnerabilities, or inefficiencies that require careful human review. Blindly accepting AI suggestions without review is dangerous.
- 2. Intellectual property and licensing:** AI models trained on open-source code may reproduce copyrighted or licensed code snippets, raising legal concerns.
- 3. Over-reliance:** Excessive dependence on AI tools can erode a developer's fundamental programming skills and understanding of underlying concepts.

9.4 AI-Assisted Software Development

Challenges and risks:

- 4. Security risks:** AI-generated code may introduce security vulnerabilities if the model has learned insecure coding patterns from training data.
- 5. Hallucination:** AI models can generate plausible-looking but incorrect code, invent non-existent API functions, or produce logically flawed algorithms.

9.5 Low-Code/No-Code, Green Software Engineering

Low-Code/No-Code Development

Low-code and no-code platforms are development environments that allow users to create applications through graphical user interfaces, visual modeling, and drag-and-drop components rather than writing traditional hand-coded programs.

Low-code platforms provide a visual development environment where most of the application can be built using visual tools (drag-and-drop UI builders, workflow designers, form builders), but developers can still write custom code when needed for complex logic, integrations, or customizations. Low-code platforms are aimed at professional developers who want to accelerate development. Examples include Microsoft Power Apps, OutSystems, and Mendix.

9.5 Low-Code/No-Code, Green Software Engineering

No-code platforms take this further by eliminating the need for any programming knowledge entirely. Applications are built purely through visual interfaces, pre-built templates, and configuration. No-code platforms target business users and non-technical professionals ("citizen developers") who want to automate workflows or build simple applications without depending on IT departments.

Examples include Airtable, Bubble, and Zapier.

Benefits of low-code/no-code:

- 1. Faster development:** Applications can be built in days or weeks instead of months, dramatically reducing time-to-market.
- 2. Lower cost:** Reduced need for specialized development resources lowers project costs.

9.5 Low-Code/No-Code, Green Software Engineering

Benefits of low-code/no-code:

3. Democratization of development: Business users and domain experts can build applications directly, reducing the backlog on IT departments and bridging the gap between business needs and technology solutions.

4. Rapid prototyping: Ideas can be quickly prototyped and validated before committing to full-scale development.

5. Reduced maintenance burden: Platform updates, security patches, and infrastructure management are handled by the platform provider.

9.5 Low-Code/No-Code, Green Software Engineering

Limitations of low-code/no-code:

- 1. Limited customization:** Complex business logic, unique UI requirements, or advanced integrations may be difficult or impossible to implement within the constraints of the platform.
- 2. Vendor lock-in:** Applications built on proprietary platforms are tied to that vendor. Migrating to another platform or to custom code can be costly and difficult.
- 3. Scalability constraints:** Applications may not scale well for high-traffic, enterprise-grade workloads that demand fine-tuned performance optimization.

9.5 Low-Code/No-Code, Green Software Engineering

Limitations of low-code/no-code:

4. Security and governance: Less visibility into underlying code makes security auditing and compliance more challenging.

5. Technical debt: Poorly designed visual workflows ("visual spaghetti") can become as difficult to maintain as poorly written code.

9.5 Low-Code/No-Code, Green Software Engineering

Green Software Engineering

Green software engineering is an emerging discipline that focuses on designing, developing, deploying, and managing software systems in ways that minimize their environmental impact, primarily their carbon emissions and energy consumption. It recognizes that the ICT sector contributes significantly to global carbon emissions through energy-intensive data centers, network infrastructure, and end-user devices. The Green Software Foundation (GSF), established in 2021 by organizations including Microsoft, GitHub, Accenture, and ThoughtWorks, defines green software engineering through a set of core principles.

9.5 Low-Code/No-Code, Green Software Engineering

Principles of green software engineering:

1. Energy efficiency:

Build applications that consume the least amount of energy possible. This involves writing efficient algorithms, minimizing unnecessary computation, reducing idle resource consumption, and optimizing data processing. For example, choosing an $O(n \log n)$ algorithm over an $O(n^2)$ algorithm for large datasets reduces both execution time and energy consumption.

9.5 Low-Code/No-Code, Green Software Engineering

Principles of green software engineering:

2. Carbon awareness:

Be aware of the carbon intensity of the electricity grid at different times and locations. Schedule energy-intensive workloads (batch processing, model training, large builds) to run when and where the grid is powered by cleaner energy sources (e.g., during sunny or windy periods, or in regions with high renewable energy penetration).

9.5 Low-Code/No-Code, Green Software Engineering

Principles of green software engineering:

3. Hardware efficiency:

Maximize the utilization of existing hardware. Underutilized servers waste embodied carbon (the carbon emitted during manufacturing). Design software to run efficiently on existing hardware rather than requiring frequent upgrades. Extend the useful life of devices and infrastructure.

4. Measurement:

Quantify the carbon emissions and energy consumption of software systems. What is not measured cannot be improved. Use tools and frameworks to track the carbon footprint of applications, deployments, and infrastructure.

9.5 Low-Code/No-Code, Green Software Engineering

Practices for green software engineering:

- 1. Efficient coding:** Write optimized code that minimizes CPU cycles, memory usage, and I/O operations. Avoid unnecessary polling, redundant computations, and resource-intensive background processes.
- 2. Right-sizing infrastructure:** Provision only the resources needed. Use auto-scaling to match resource allocation to actual demand rather than provisioning for peak capacity at all times.
- 3. Demand shaping:** Design applications to shift workloads to times when carbon-free energy is available. Use event-driven architectures rather than always-on processes.

9.5 Low-Code/No-Code, Green Software Engineering

Practices for green software engineering:

4. Data efficiency: Store and transfer only necessary data. Compress data, archive infrequently accessed data to cold storage, and avoid excessive logging or telemetry that wastes storage and processing.

5. Sustainable architecture: Choose architectures that inherently reduce waste. This includes serverless computing where functions execute only when triggered, efficient caching strategies, and CDNs that reduce network traffic.

9.5 Low-Code/No-Code, Green Software Engineering

Green software engineering treats sustainability as a non-functional requirement alongside performance, security, and reliability. This integrates environmental responsibility into the software development lifecycle rather than treating it as an afterthought.